```
import kagglehub  # This line fixes the NameError
import os
from PIL import Image
import matplotlib.pyplot as plt

# 1. Download the latest version of the dataset
path = kagglehub.dataset_download("dataclusterlabs/indian-number-plates-dataset")
print("Path to dataset files:", path)

# 2. Get list of all images in the downloaded folder
image_extensions = ('.jpg', '.jpeg', '.png')
all_images = []

for root, dirs, files in os.walk(path):
    for file in files:
        if file.lower().endswith(image_extensions):
            all_images.append(os.path.join(root, file))

# 3. Visualize a few samples
if all_images:
    plt.figure(figsize=(12, 8))
    for i in range(min(3, len(all_images))):
        img = Image.open(all_images[i])
        plt.subplot(1, 3, i + 1)
        plt.imshow(img)
        plt.title(f"Sample {i+1}")
        plt.axis('off')
    plt.show()
else:
    print("No images found in the path. Please verify the download.")
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest ve
Downloading from https://www.kaggle.com/api/v1/datasets/download/dataclusterlabs/indian-number-plates-dataset?dataset_version_nu
100%|██████████| 145M/145M [00:05<00:00, 27.0MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/dataclusterlabs/indian-number-plates-dataset/versions/3
```

```
import os
import cv2
import pickle
import kagglehub
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split

# 1. Download Dataset
path = kagglehub.dataset_download("dataclusterlabs/indian-number-plates-dataset")

# 2. Define CNN Architecture (Character Recognizer)
def create_cnn_model(num_classes=36): # 10 digits + 26 letters
    model = models.Sequential([
        layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 1)),
```

```
        layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 1)),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return model

# 3. Preprocessing Functions
def segment_characters(plate_img):
    """Detects and crops individual characters from the plate bounding box."""
    gray = cv2.cvtColor(plate_img, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    thresh = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)[1]

    # Find contours within the plate
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    char_images = []

    for cnt in contours:
        x, y, w, h = cv2.boundingRect(cnt)
        # Filter for typical character dimensions
        if 10 < w < 80 and 20 < h < 100:
            roi = thresh[y:y+h, x:x+w]
            roi = cv2.resize(roi, (32, 32))
            char_images.append(roi.reshape(32, 32, 1) / 255.0)

    return char_images

# 4. Main Training Pipeline
def run_pipeline():
    # Placeholder for loading your specific bounding box annotations
    # Use [OpenCV Slicing](https://docs.opencv.org) to isolate numbers
    # Example: plate_roi = image[y:y+h, x:x+w]

    print("Building model...")
    model = create_cnn_model()

    # Generate mock data for demonstration (Replace with segments from all_images)
    X = np.random.rand(100, 32, 32, 1)
    y = np.random.randint(0, 36, 100)

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

    print("Training Deep Learning model...")
    history = model.fit(X_train, y_train, epochs=5, validation_data=(X_test, y_test), verbose=1)

    # Calculate Final Accuracy
    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
    print(f"\nFinal Model Accuracy: {accuracy * 100:.2f}%")

    # 5. Pickle the Model
    # Note: Keras models are best saved as .h5, but we can pickle parameters
    model_export = {
        "weights": model.get_weights(),
        "accuracy": accuracy,
        "classes": "0-9, A-Z"
    }

    pickle_file = "indian_plate_number_model.pkl"
    with open(pickle_file, 'wb') as f:
        pickle.dump(model_export, f)

    print(f"Model and accuracy results pickled to: {os.path.abspath(pickle_file)}")

if __name__ == "__main__":
    run_pipeline()
```

```
Using Colab cache for faster access to the 'indian-number-plates-dataset' dataset.
Building model...
Training Deep Learning model...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 2s 182ms/step - accuracy: 0.0102 - loss: 3.6262 - val_accuracy: 0.1000 - val_loss: 3.5978
Epoch 2/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step - accuracy: 0.0688 - loss: 3.5140 - val_accuracy: 0.0500 - val_loss: 3.6756
Epoch 3/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step - accuracy: 0.0727 - loss: 3.4663 - val_accuracy: 0.0500 - val_loss: 3.6911
Epoch 4/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step - accuracy: 0.0805 - loss: 3.4519 - val_accuracy: 0.0500 - val_loss: 3.6914
Epoch 5/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step - accuracy: 0.0727 - loss: 3.4201 - val_accuracy: 0.0500 - val_loss: 3.7038

Final Model Accuracy: 5.00%
Model and accuracy results pickled to: /content/indian_plate_number_model.pkl
```

```
import os
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# --- 1. CONFIGURATION ---
IMG_SIZE = 32
DATASET_PATH = "/root/.cache/kagglehub/datasets/dataclusterlabs/indian-number-plates-dataset/versions/1"

# --- 2. IMPROVED PREPROCESSING ---
def preprocess_image(img_path):
    """Loads image, converts to grayscale, and resizes."""
    img = cv2.imread(img_path)
    if img is None:
        return None
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Use adaptive thresholding to handle lighting variations
    thresh = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                   cv2.THRESH_BINARY_INV, 11, 2)
    resized = cv2.resize(thresh, (IMG_SIZE, IMG_SIZE))
    return resized.reshape(IMG_SIZE, IMG_SIZE, 1) / 255.0

# --- 3. ROBUST CNN ARCHITECTURE ---
def create_optimized_model(num_classes):
    model = models.Sequential([
        # Layer 1: Feature Extraction
        layers.Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(IMG_SIZE, IMG_SIZE, 1)),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        # Layer 2: Deeper features
        layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        # Layer 3: Global patterns
        layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        # Dense Layers
        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.4),  # Dropout prevents overfitting
        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# --- 4. DATA LOADING LOGIC ---
def load_real_data():
    """
    Scans the dataset folder for images.
    Note: You should organize characters into subfolders (0-9, A-Z).
    """
    X = []
    y = []
```

```
    # This logic assumes images are in folders named by their character
    # If your dataset is just raw plate images, you'd need a CSV to crop them first.
    # For now, we will simulate a character-level loop.
    for root, dirs, files in os.walk(DATASET_PATH):
        for file in files[:1000]: # Limit for speed
            if file.endswith(('.jpg', '.png', '.jpeg')):
                img_path = os.path.join(root, file)
                img_data = preprocess_image(img_path)
                if img_data is not None:
                    X.append(img_data)
                    # Use the folder name as the label
                    label = os.path.basename(root)
                    y.append(label)

    return np.array(X), np.array(y)

# --- 5. EXECUTION PIPELINE ---
def run_pipeline():
    print("Step 1: Loading and Preprocessing Real Data...")
    X, y_raw = load_real_data()

    if len(X) == 0:
        print("Error: No images found. Checking for mock fallback...")
        # Fallback to dummy data ONLY if dataset path is empty/wrong
        X = np.random.rand(500, 32, 32, 1)
        y_raw = np.random.randint(0, 36, 500)

    # Encode labels (A-Z, 0-9) to integers
    le = LabelEncoder()
    y = le.fit_transform(y_raw)
    num_classes = len(np.unique(y))

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    print(f"Step 2: Building Model for {num_classes} classes...")
    model = create_optimized_model(num_classes)

    # Use EarlyStopping to stop when accuracy stops improving
    early_stop = callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

    print("Step 3: Training...")
    model.fit(X_train, y_train,
              epochs=30,
              batch_size=32,
              validation_data=(X_test, y_test),
              callbacks=[early_stop])

    print("\nStep 4: Final Evaluation...")
    loss, accuracy = model.evaluate(X_test, y_test)
    print(f"--- Final Accuracy: {accuracy * 100:.2f}% ---")

    # Step 5: Save (Keras .keras format is safer than pickle)
    model.save("indian_plate_character_model.keras")
    print("Model saved as 'indian_plate_character_model.keras'")

if __name__ == "__main__":
    run_pipeline()
```

```
Step 1: Loading and Preprocessing Real Data...
Error: No images found. Checking for mock fallback...
Step 2: Building Model for 36 classes...
Step 3: Training...
Epoch 1/30
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
13/13 ━━━━━━━━━━━━━━━━━━━━ 9s 168ms/step - accuracy: 0.0135 - loss: 5.4247 - val_accuracy: 0.0100 - val_loss: 3.6302
Epoch 2/30
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 102ms/step - accuracy: 0.0756 - loss: 3.5183 - val_accuracy: 0.0100 - val_loss: 4.4569
Epoch 3/30
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 100ms/step - accuracy: 0.1065 - loss: 3.3863 - val_accuracy: 0.0100 - val_loss: 5.9156
Epoch 4/30
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 98ms/step - accuracy: 0.1637 - loss: 3.0700 - val_accuracy: 0.0100 - val_loss: 6.9104
Epoch 5/30
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 96ms/step - accuracy: 0.2258 - loss: 2.7409 - val_accuracy: 0.0100 - val_loss: 7.5275
Epoch 6/30
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 99ms/step - accuracy: 0.3606 - loss: 2.3329 - val_accuracy: 0.0100 - val_loss: 7.9269

Step 4: Final Evaluation...
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.0092 - loss: 3.6305
--- Final Accuracy: 1.00% ---
Model saved as 'indian_plate_character_model.keras'
```

```
!apt install tesseract-ocr
!pip install pytesseract
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tesseract-ocr is already the newest version (4.1.1-2.1build1).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
Collecting pytesseract
  Downloading pytesseract-0.3.13-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from pytesseract) (25.0)
Requirement already satisfied: Pillow>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from pytesseract) (11.3.0)
Downloading pytesseract-0.3.13-py3-none-any.whl (14 kB)
Installing collected packages: pytesseract
Successfully installed pytesseract-0.3.13
```

```
import os
import cv2
import kagglehub
import pytesseract # Use Tesseract for high accuracy immediately

# 1. Download and get the EXACT path
print("Downloading dataset...")
path = kagglehub.dataset_download("dataclusterlabs/indian-number-plates-dataset")
print(f"Dataset location: {path}")

# 2. Function to extract text using Tesseract OCR
def extract_plate_text(image_path):
    img = cv2.imread(image_path)
    if img is None: return None

    # Pre-processing for better OCR accuracy
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)[1]

    # Use Tesseract to find text (much higher accuracy than 1%)
    # Install: !apt install tesseract-ocr
    text = pytesseract.image_to_string(gray, config='--psm 7')
    return text.strip()

# 3. Process the actual images
image_files = []
for root, dirs, files in os.walk(path):
    for file in files:
        if file.lower().endswith(('.png', '.jpg', '.jpeg')):
            image_files.append(os.path.join(root, file))

print(f"Found {len(image_files)} real images!")

# Test on the first 5 images
for i in range(5):
    plate_text = extract_plate_text(image_files[i])
    print(f"Image {i+1} Result: {plate_text}")
```

```
Downloading dataset...
Using Colab cache for faster access to the 'indian-number-plates-dataset' dataset.
Dataset location: /kaggle/input/indian-number-plates-dataset
Found 47 real images!
Image 1 Result:
Image 2 Result:
Image 3 Result:
Image 4 Result:
Image 5 Result: | |
```

```
# 1. INSTALL DEPENDENCIES
!apt install tesseract-ocr
!pip install pytesseract

import os
import cv2
import kagglehub
```

```
import pytesseract
import numpy as np
from google.colab.patches import cv2_imshow

# 2. DOWNLOAD DATASET
path = kagglehub.dataset_download("dataclusterlabs/indian-number-plates-dataset")

def process_and_read(image_path):
    # Load Image
    img = cv2.imread(image_path)
    if img is None: return ""

    # Resize for consistency
    img = cv2.resize(img, (620, 480))

    # Convert to Gray and Filter Noise
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.bilateralFilter(gray, 11, 17, 17) # Blur but keep edges sharp

    # Find Edges
    edged = cv2.Canny(gray, 30, 200)

    # Find Contours (Shapes)
    contours, _ = cv2.findContours(edged.copy(), cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
    contours = sorted(contours, key=cv2.contourArea, reverse=True)[:10]

    screenCnt = None
    for c in contours:
        # Approximate the contour
        peri = cv2.arcLength(c, True)
        approx = cv2.approxPolyDP(c, 0.018 * peri, True)
        # If our contour has four points, it's likely our plate
        if len(approx) == 4:
            screenCnt = approx
            break

    if screenCnt is not None:
        # Mask everything except the plate
        mask = np.zeros(gray.shape, np.uint8)
        new_image = cv2.drawContours(mask, [screenCnt], 0, 255, -1)
        new_image = cv2.bitwise_and(img, img, mask=mask)

        # Crop to the plate
        (x, y) = np.where(mask == 255)
        (topx, topy) = (np.min(x), np.min(y))
        (bottomx, bottomy) = (np.max(x), np.max(y))
        cropped = gray[topx:bottomx+1, topy:bottomy+1]

        # OCR on the cropped plate
        config = '--psm 7 -c tessedit_char_whitelist=0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ'
        text = pytesseract.image_to_string(cropped, config=config)
        return text.strip()

    return "Plate Not Detected"

# 3. RUN ON DATASET
image_files = []
for root, dirs, files in os.walk(path):
    for file in files:
        if file.lower().endswith(('.png', '.jpg', '.jpeg')):
            image_files.append(os.path.join(root, file))

print(f"Found {len(image_files)} images. Processing...")

for i in range(min(10, len(image_files))):
    result = process_and_read(image_files[i])
    print(f"Image {i+1}: {result}")
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tesseract-ocr is already the newest version (4.1.1-2.1build1).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
Requirement already satisfied: pytesseract in /usr/local/lib/python3.12/dist-packages (0.3.13)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from pytesseract) (25.0)
Requirement already satisfied: Pillow>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from pytesseract) (11.3.0)
Using Colab cache for faster access to the 'indian-number-plates-dataset' dataset.
```

```
Found 47 images. Processing...
Image 1: Plate Not Detected
Image 2:
Image 3: A
Image 4:
Image 5: Plate Not Detected
Image 6: P8 4A I
Image 7:
Image 8: Plate Not Detected
Image 9:
Image 10: Plate Not Detected
```

```
!pip install python-Levenshtein
```

```
Collecting python-Levenshtein
  Downloading python_levenshtein-0.27.3-py3-none-any.whl.metadata (3.9 kB)
Collecting Levenshtein==0.27.3 (from python-Levenshtein)
  Downloading levenshtein-0.27.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (3.7 kB)
Collecting rapidfuzz<4.0.0,>=3.9.0 (from Levenshtein==0.27.3->python-Levenshtein)
  Downloading rapidfuzz-3.14.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (12 kB)
Downloading python_levenshtein-0.27.3-py3-none-any.whl (9.5 kB)
Downloading levenshtein-0.27.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (153 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 153.3/153.3 kB 3.3 MB/s eta 0:00:00
Downloading rapidfuzz-3.14.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (3.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 31.7 MB/s eta 0:00:00
Installing collected packages: rapidfuzz, Levenshtein, python-Levenshtein
Successfully installed Levenshtein-0.27.3 python-Levenshtein-0.27.3 rapidfuzz-3.14.3
```

```
import pytesseract
import cv2
import os

# 1. Install necessary library for accuracy math
# !pip install python-Levenshtein

from Levenshtein import distance as lev_distance

def calculate_accuracy(original, predicted):
    """Calculates accuracy based on character error distance."""
    if not original: return 0.0
    # Clean strings: remove spaces and make uppercase
    original = "".join(original.split()).upper()
    predicted = "".join(predicted.split()).upper()

    # Calculate character-level distance (errors)
    errors = lev_distance(original, predicted)

    # Calculate accuracy
    acc = max(0, (len(original) - errors) / len(original))
    return acc * 100

# Example Test Case (Use this inside your loop)
ground_truth_list = ["MH12DE1433", "DL3CAS2910"] # Replace with real labels from dataset
detected_list = [] # Your OCR results go here

# Mock loop for demonstration
for i, true_val in enumerate(ground_truth_list):
    # result = process_and_read(image_paths[i])  <-- Your previous OCR function
    result = "MH12DE1438" # Example OCR result (1 error: 3 vs 8)

    acc = calculate_accuracy(true_val, result)
    print(f"Image {i+1}:")
    print(f"  Actual:    {true_val}")
    print(f"  Predicted: {result}")
    print(f"  Accuracy:  {acc:.2f}%")
```

```
Image 1:
  Actual:    MH12DE1433
  Predicted: MH12DE1438
  Accuracy:  90.00%
Image 2:
  Actual:    DL3CAS2910
  Predicted: MH12DE1438
  Accuracy:  0.00%
```

Start coding or generate with AI.